

Curso Tecnologia em ADS
Sistemas Operacionais

Laboratório: Mecanismos de Comunicação entre Processos

Professor: Eraldo Silveira e Silva Material elaborado com suporte do ChatGPT

1 Objetivo

Explorar experimentalmente os mecanismos de comunicação entre processos em sistemas Linux: **pipes não nomeados**, **FIFOs nomeadas**, **filas de mensagens POSIX** e **memória compartilhada POSIX**. Os exercícios abordam criação e uso dos mecanismos, operações bloqueantes e não bloqueantes, limites de capacidade, concorrência, sincronização e espera por eventos.

2 Orientações gerais

O professor definirá a abrangência das atividades a serem realizadas. Para cada exercício selecionado, deverão ser implementados programas em linguagem C, demonstrada sua execução e explicado o comportamento observado. Identificar os processos envolvidos, as primitivas utilizadas, os descritores ou regiões compartilhadas, as condições de bloqueio e desbloqueio, os valores de retorno e os erros relevantes. Usar mensagens de execução para evidenciar o comportamento dos programas. **É permitido utilizar inteligência artificial como apoio.** Entretanto, todos os programas deverão ser compreendidos integralmente por seus autores. **Os arquivos de código-fonte C entregues não poderão conter comentários.** A explicação do código deverá ser realizada nos slides e oralmente, inclusive para trechos indicados pelo professor.

3 Parte I – Pipes não nomeados e FIFOs

Exercício 1 – Mini Pipeline

Implemente um programa equivalente ao comando:

```
1 ls | wc -l
```

O programa deve criar um pipe, criar dois processos filhos, redirecionar a saída do primeiro para o pipe, redirecionar a entrada do segundo a partir do pipe e executar os comandos com uma função da família `exec`.

O programa deve aguardar o término dos filhos utilizando uma função da família `wait`.

Explique: o papel de `pipe()`, o uso de `dup2()` e a necessidade de fechar descritores.

Exercício 2 – Leitura Bloqueante

Implemente um programa em C que crie um pipe e um processo filho.

O processo filho deverá executar `read()` para receber uma mensagem do pai. O processo pai deverá aguardar 5 segundos e, em seguida, escrever a mensagem "`IFSC São José`" no pipe.

O filho deverá apresentar a mensagem recebida na saída padrão.

Observe e explique: por que `read()` bloqueia e em que momento a operação é desbloqueada.

Exercício 3 – Leitura Não Bloqueante

Modifique o programa do Exercício 2 para que o processo filho realize a leitura do pipe em modo não bloqueante, utilizando `fcntl()` e a opção `O_NONBLOCK`.

O processo pai deverá aguardar 5 segundos antes de escrever a mensagem `"IFSC São José"` no pipe.

O filho deverá tentar ler o pipe periodicamente, com intervalo de 1 segundo entre as tentativas, até receber a mensagem.

O programa deverá apresentar uma indicação sempre que uma tentativa de leitura não encontrar dados disponíveis.

Explique: como distinguir a ausência temporária de dados, o fim do pipe (EOF) e um erro de leitura.

Exercício 4 – Fim de Arquivo (EOF)

Implemente um programa em que o escritor envia dados e fecha o pipe; o leitor deverá ler até detectar EOF.

Explique: quando `read()` retorna zero e o papel do fechamento de todas as extremidades de escrita.

Exercício 5 – Monitoramento de Pipes com `select()`

Implemente um programa em C que crie dois pipes e três processos filhos: dois escritores e um receptor.

O primeiro escritor deverá enviar cinco mensagens pelo primeiro pipe, com intervalo de 3 segundos entre os envios. O segundo escritor deverá enviar cinco mensagens pelo segundo pipe, com intervalo de 8 segundos entre os envios.

Cada mensagem deverá identificar o processo que a enviou.

O processo receptor deverá utilizar `select()` para monitorar simultaneamente os dois pipes. Sempre que receber dados, deverá apresentar a mensagem recebida e identificar o escritor.

Caso nenhuma mensagem seja recebida durante 5 segundos, o receptor deverá apresentar a mensagem `"Aguardando mensagens..."` e continuar monitorando os pipes.

Após o término de cada escritor, o receptor deverá detectar EOF e deixar de monitorar o respectivo pipe. O processo pai deverá aguardar o término dos três filhos.

Explique: como `select()` identifica os descritores prontos para leitura, o funcionamento do timeout e o tratamento de EOF.

Exercício 6 – FIFO Nomeada (atividade adicional)

Crie uma FIFO usando `mkfifo()`. Implemente dois programas independentes: um escritor e um leitor, que se comuniquem por meio do nome da FIFO no sistema de arquivos.

Observe e explique: o comportamento de `open()` quando não existe outra extremidade aberta, a diferença entre a entrada da FIFO no sistema de arquivos e os dados armazenados no núcleo, e o efeito de `unlink()`.

Exercício 7 – Duas FIFOs e `select()` (atividade adicional)

Crie duas FIFOs. Dois escritores independentes deverão produzir dados em intervalos diferentes; um processo leitor deverá acompanhar ambas usando `select()` com timeout.

Explique: como identificar qual descritor está pronto, o significado de prontidão quando ocorre EOF e os cuidados necessários com a abertura e o fechamento das FIFOs.

4 Parte II – Filas de mensagens POSIX

Primitivas e atributos

Nos exercícios desta parte, considere as primitivas `mq_open()`, `mq_close()`, `mq_unlink()`, `mq_send()`, `mq_receive()`, `mq_timedsend()`, `mq_timedreceive()`, `mq_getattr()` e `mq_setattr()`.

Quando necessário, use `struct mq_attr` para configurar capacidade, tamanho máximo de mensagem e modo não bloqueante.

Exercício 1 – Uso Básico da Fila: Bloqueante, Não Bloqueante e Timeout na Recepção

Implemente dois processos:

- um processo transmissor;
- um processo receptor.

O transmissor deve enviar uma mensagem textual simples, como por exemplo:

```
"Temperatura=29.5"
```

O receptor deve abrir a mesma fila e receber essa mensagem, exibindo-a na tela.
Este exercício deve ser dividido em três partes:

1. **Recepção bloqueante:** o receptor chama `mq_receive()` antes do transmissor enviar a mensagem.
2. **Recepção não bloqueante:** o receptor abre a fila em modo não bloqueante e tenta receber uma mensagem quando a fila está vazia.
3. **Recepção com timeout:** o receptor utiliza `mq_timedreceive()` com um tempo limite, enquanto o transmissor envia a mensagem somente após esse tempo expirar.

Explique:

- o papel de `mq_open()`;
- a diferença entre abrir a fila para escrita e para leitura;
- por que a recepção pode bloquear;
- o que muda no modo não bloqueante;
- o significado de um retorno por expiração de tempo;
- a diferença entre:
  - bloqueio indefinido;
  - retorno imediato;
  - espera com tempo limite.

Exercício 2 – Capacidade da Fila, Tamanho da Mensagem e Timeout no Envio

Crie uma fila com capacidade pequena, por exemplo 3 mensagens, utilizando `struct mq_attr`.

Defina também explicitamente o tamanho máximo de cada mensagem.

Este exercício deve explorar:

a) **Capacidade máxima da fila:** um transmissor deve enviar mensagens continuamente, enquanto o receptor demora para consumi-las.

b) **Envio bloqueante:** observe o momento em que `mq_send()` bloqueia porque a fila está cheia.

c) **Envio com timeout:** substitua o envio por `mq_timedsend()` e observe o que ocorre quando não há espaço disponível antes do tempo limite.

d) **Tamanho da mensagem:** tente enviar uma mensagem maior que o tamanho máximo configurado para a fila.

Explique:

- o papel de `mq_maxmsg`;
- o papel de `mq_msgsize`;
- o que acontece quando a fila enche;
- o que causa o desbloqueio do transmissor;
- o que muda entre:
  - envio bloqueante;
  - envio com tempo limite;
  - falha por mensagem grande demais.

Exercício 3 – Concorrência com Múltiplos Processos

Implemente duas situações distintas.

Parte A – Múltiplos transmissores e um receptor Implemente um programa com:

- dois processos transmissores;
- um processo receptor.

Cada transmissor deve enviar mensagens identificadas com sua origem, por exemplo:

```
3 "TX1 - leitura 10"
4 "TX2 - leitura 15"
```

Observe:

- a ordem em que as mensagens chegam;
- possíveis variações entre execuções;
- se cada mensagem é recebida inteira ou pode ser misturada com outra.

Parte B – Um transmissor e múltiplos receptores Implemente um programa com:

- um processo transmissor;
- dois processos receptores.

O transmissor deve enviar várias mensagens numeradas.
Observe:

- como as mensagens são distribuídas entre os receptores;
- se uma mesma mensagem pode ser entregue duas vezes;
- se os receptores competem entre si pelas mensagens disponíveis.

Explique:

- por que as mensagens são preservadas como unidades discretas;
- por que não ocorre difusão automática das mensagens para todos os receptores;
- a diferença entre concorrência entre transmissores e concorrência entre receptores.